Design Documentation

Team 52 — Clue! (G6046 Software Engineering)

1. Introduction

This document describes the design of the Team 52 Clue! project. It covers both high level architecture and low level class structure, and explains the key design decisions made across the three sprints. The design evolved incrementally alongside the implementation, starting with a core model and console prototype before extending to a JavaFX GUI.

2. High-Level Architecture

2.1 System Overview

The system is structured as a layered application with three main layers:

Layer	Responsibility
Model layer	Core game rules, state and data: Board, Game, Players, Cards, Dice, MurderEnvelope
Controller layer	Coordinates turn flow between model and GUI: GameController
View layer	JavaFX rendering and user interaction: BoardView, SidebarView, SetupView, NotebookView

The model layer has no dependency on the view or controller layers. This allowed the console prototype to be developed and tested independently of the GUI, and made it possible to add JavaFX in Sprint 3 without rewriting the rules layer.

2.2 Package Structure

Package	Contents
com.cluegame.model	Game, Board, Square, Room, Dice, Suggestion, Accusation
com.cluegame.cards	Card (abstract), SuspectCard, WeaponCard, RoomCard, Deck, MurderEnvelope
com.cluegame.players	Player (abstract), HumanPlayer, AIPlayer, DetectiveNotepad
com.cluegame.gui	ClueGameApp, GameController, BoardView, SidebarView, SetupView, NotebookView

3. Low Level Class Design

3.1 Class Descriptions

Model Package

Class	Responsibility

Game	Top level controller: setup, dealing, turn order, suggestions, accusations and overall state
Board	25x24 grid of Square objects: room positions, door detection, movement validation, secret passages
Square	Single grid cell with Type enum (CORRIDOR, ROOM, BLOCKED, START) and occupancy flag
Room	Room name, door squares, secret passage target and occupant list
Dice	Fair two die roll using java.util.Random
Suggestion	Value object holding one SuspectCard, WeaponCard and RoomCard
Accusation	Extends Suggestion: semantic distinction indicating a check against the murder envelope

Cards Package

Class	Responsibility
Card (abstract)	Base class with name and CardType enum (SUSPECT, WEAPON, ROOM)
SuspectCard	Concrete card for one of the 6 suspects
WeaponCard	Concrete card for one of the 6 weapons
RoomCard	Concrete card for one of the 9 rooms
Deck	Creates all 21 cards, supports shuffle and round-robin dealing
MurderEnvelope	Holds one hidden card of each type: seal() stores the solution, verify() checks accusations

Players Package

Class	Responsibility
Player (abstract)	Stores name, token, position, hand, active flag and current room: defines abstract turn behaviour
HumanPlayer	Player driven by GUI input through GameController
AIPlayer	Autonomous player: moves toward rooms using Manhattan distance, uses DetectiveNotepad for deduction
DetectiveNotepad	Tracks seen cards and deduces the likely solution for AI decision making

GUI Package

Class	Responsibility
ClueGameApp	JavaFX Application entry point: creates the primary Stage and Scene
GameController	Mediates between Game model and JavaFX views: coordinates all turn phases
BoardView	Renders board image background, player tokens, non-player suspect pieces and weapon markers using Canvas overlay

SidebarView	Displays current player info, hand of cards, turn controls, dice result and scrollable game log
SetupView	Handles player count selection, name entry and character assignment before game start
NotebookView	Displays detective notebook with manual checkboxes for suspects, weapons and rooms

3.2 Key Relationships Between Classes

Inheritance

- Card is the parent of SuspectCard, WeaponCard and RoomCard.
- Player is the parent of HumanPlayer and AIPlayer.

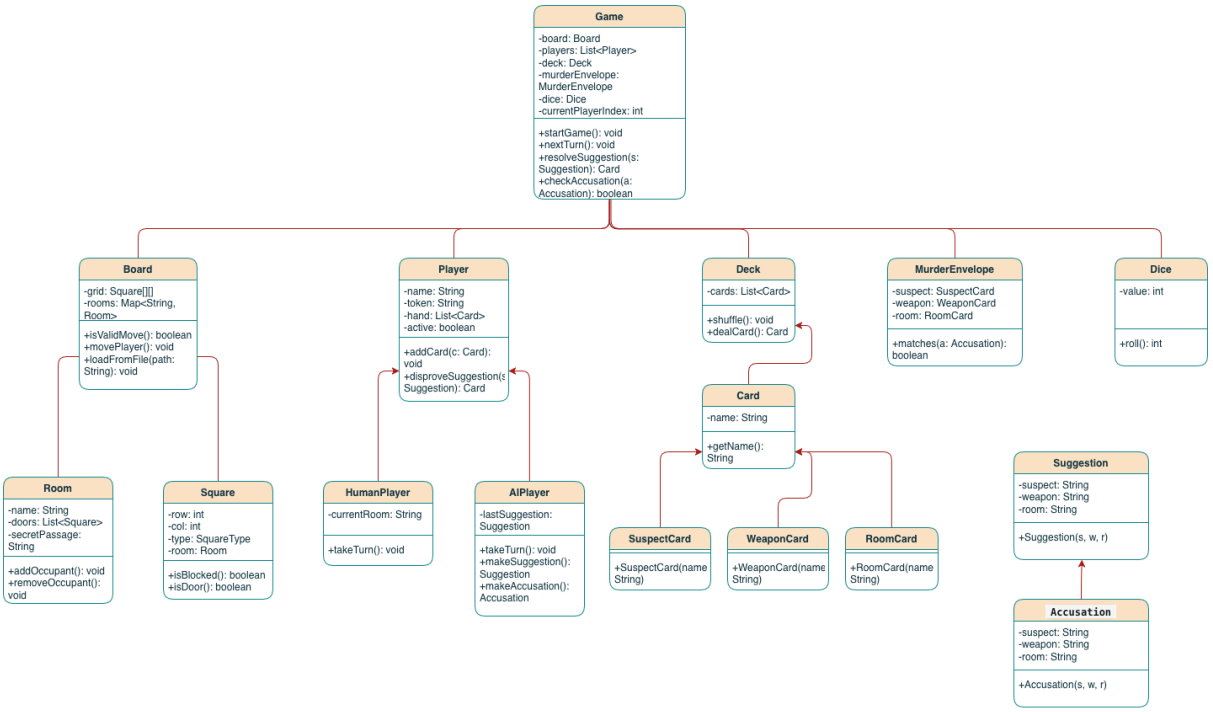
Composition / Ownership

- Game owns the Board, Dice, active players and overall match state.
- Deck creates the cards used by the game.
- MurderEnvelope stores the hidden solution cards.
- Board contains Square objects and room definitions.

Associations

- Players interact with the Board through their row/column position and room membership.
- Suggestions and accusations are created during gameplay and checked against the game state.
- GameController uses Game as the main source of truth and coordinates with BoardView and SidebarView.

Figure 1. Core game/model class diagram



4. Key Design Decisions

4.1 Separation of Model and GUI

One of the most important design decisions was to keep the core game logic out of the JavaFX views. This gave several benefits:

- Model classes could be tested directly using JUnit without any GUI dependency.
- The console version remained usable while GUI work was in progress.
- Late GUI changes did not require the rules layer to be rewritten.
- Controller logic could evolve separately from rendering code.

4.2 Use of Inheritance

Inheritance was used where there was a genuine shared abstraction: cards and players. This made the code easier to understand and aligned with the coursework emphasis on OO principles such as inheritance and polymorphism.

4.3 Grid-Based Board Representation

The board was represented as a 2D grid because this made it straightforward to validate movement, mark blocked regions, identify doors and starts, keep room logic explicit, and connect logical positions to GUI rendering. Although the board image introduced alignment challenges later, the underlying grid model remained a strong design choice for the rules layer.

4.4 Controller-Mediated GUI Logic

Rather than placing game flow directly inside JavaFX views, GameController was used as a mediator between Game, BoardView and SidebarView. This improved maintainability and kept responsibilities clearer: views display state and capture input, the controller coordinates actions, and the model enforces rules.

4.5 Incremental AI Design

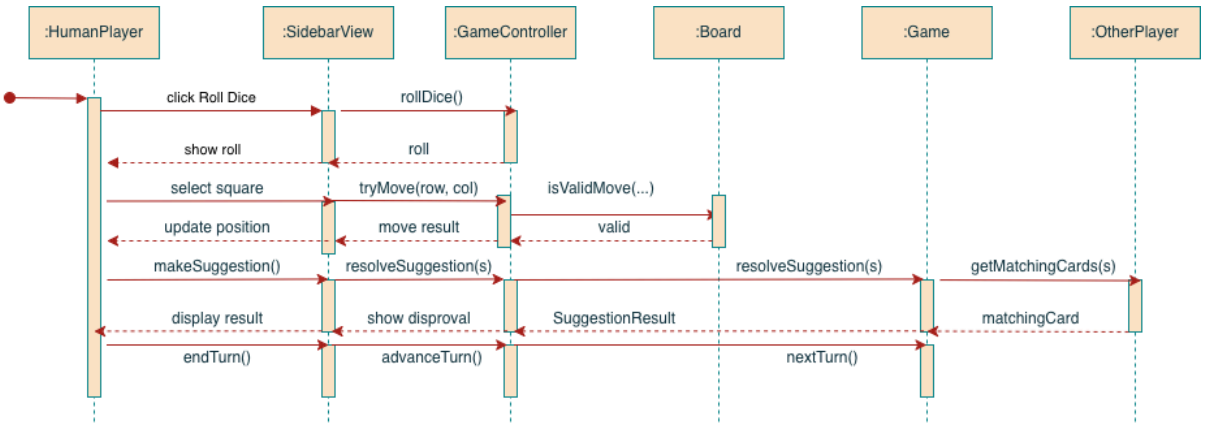
The coursework brief did not require advanced AI strength, only a working autonomous player. For that reason the initial AI was kept simple and AI sophistication was improved incrementally only where needed for playability. Correctness and completeness were prioritised over advanced deduction strategy.

5. Behavioural Design

5.1 Normal Turn Sequence

The main normal turn interaction involves SidebarView requesting an action, GameController checking and updating state, Game and Board performing rule based updates, and BoardView and SidebarView refreshing the visible state.

Figure 2. Normal turn sequence diagram



5.2 Suggestion and Disproving Sequence

Suggestions involve coordination across several layers: the current player enters a room, GameController constructs the suggestion, Game resolves suspect and weapon movement, players are checked in order for disproving, and the GUI presents the result privately to the current human player.

5.3 Accusation and Game Over

Accusations follow a similar pattern: the player creates an accusation, GameController checks it against the murder envelope, a correct accusation ends the game, and a wrong accusation eliminates the player but keeps them available for disproving future suggestions.

6. Design Validation and Evolution

The final design did not remain identical to the earliest plan. Several areas evolved during development:

Area	How it evolved
Board design	Early board logic was sufficient for the console prototype. JavaFX rendering exposed alignment and geometry issues that required several iterations to resolve.
Turn flow	Early turn handling was simpler in the console version. GUI interaction required clearer turn phases, room exit flow and AI advancement, leading to a stronger role for GameController.
Rules alignment	Character-selected starts, token colours, non-player suspects and weapon movement were tightened during Sprint 3 after playtesting.
AI design	AI began as very simple/random behaviour. Later changes focused on ensuring AI could complete turns reliably in the GUI context.

The overall architecture remained stable even as individual behaviours were corrected. This suggests the design was flexible enough to absorb changes without large-scale rewrite.

7. OO Principles Applied

Principle	How Applied
Encapsulation	Core state such as hands, positions, rooms and envelope contents is stored inside the relevant objects rather than in global structures
Inheritance	Used for cards and players where shared behaviour and identity made it appropriate
Polymorphism	Human and AI players share a common player abstraction while supporting different behaviour through overridden methods
High cohesion	Most classes have a clear central responsibility: Board for geometry, Deck for card setup, SidebarView for sidebar behaviour
Low coupling	JavaFX views do not implement the rules. The controller coordinates between packages rather than merging all logic in one place

8. Diagrams Index

Diagram	File Location
Core game/model class diagram	Diagrams/CoreModelClassDiagram.png
GUI class diagram	Diagrams/GUIClassDiagram.png
Normal turn sequence diagram	Diagrams/PlayerTurnSequenceDiagram.png
Player turn activity diagram	Diagrams/PlayerTurnActivityDiagram.png
Use case diagram	Diagrams/UseCaseDiagram_Team52.png